

Stereo Vision Depth Estimation with Object Detection on Jetson Nano

Overview

This project leverages stereo vision and object detection to estimate the depth of objects using two cameras connected to a Jetson Nano. The Jetson Nano utilizes `ssd-mobilenet-v2` for object detection and computes depth using triangulation. The project involves initializing the cameras, performing object detection, calculating depth, and displaying the results.

Dependencies

The following libraries are required:

- `opencv-python (cv2)` version 4.5.1.48 or later
- `numpy` version 1.20.1 or later
- `triangulation` (custom module for depth calculation)
- `HSV_filter` (custom module, not used directly in the provided code)
- `jetson-inference` version 0.5.1 or later (for object detection)
- `nanocamera` version 0.1.4 or later (for interfacing with the cameras)
- `jetson-utils` version 0.5.1 or later (for CUDA operations)

You can install these libraries using pip:

```
pip install opencv-python numpy
```

Ensure you have installed `jetson-inference` and `jetson-utils` following the [official Jetson Nano setup guide](#).

File Structure

- **`stereo.py`**: Main script to run the stereo vision and object detection.
- **`triangulation.py`**: Contains the `find_depth` function for depth calculation.
- **`HSV_filter.py`**: Contains HSV filtering functions (not used directly in this project).

`find_depth` Function in `triangulation.py`

This function computes the depth (z-coordinate) of an object using the disparity between the positions of the object in two images captured by a stereo camera setup.

python code:-

```
import numpy as np

def find_depth(circle_right, circle_left, frame_right,
               frame_left, baseline, f, alpha):
```

```

    # CONVERT FOCAL LENGTH f FROM [mm] TO [pixel]:
    height_right, width_right, depth_right =
frame_right.shape
    height_left, width_left, depth_left = frame_left.shape

    if width_right == width_left:
        f_pixel = (width_right * 0.5) / np.tan(alpha * 0.5 *
np.pi / 180)
    else:
        print('Left and right camera frames do not have the
same pixel width')
        return None

    x_right = circle_right[0]
    x_left = circle_left[0]

    # CALCULATE THE DISPARITY:
    disparity = x_left - x_right # Displacement between left
and right frames [pixels]

    if disparity == 0:
        print('Disparity is zero, cannot calculate depth')
        return None

    # CALCULATE DEPTH z:
    zDepth = (baseline * f_pixel) / disparity # Depth in
[cm]

    return abs(zDepth)

```

Main Script: Stereo.py

This script initializes the cameras, performs object detection, calculates depth, and displays the results.

python code:-

```

import sys
import cv2
import numpy as np
import triangulation as tri
import time
from jetson_inference import detectNet
import jetson.utils
import nanocamera as nano

```

```

# Initialize Jetson inference network
net = detectNet("ssd-mobilenet-v2", threshold=0.5)

# Initialize nanoCamera for left and right cameras
fps = 60 # Increase FPS value
cap_right = nano.Camera(flip=2, width=640, height=480,
fps=fps)
cap_left = nano.Camera(device_id=1, flip=2, width=640,
height=480, fps=fps)

# Stereo camera parameters
B = 9 # Baseline [cm]
f = 2 # Focal length [mm]
alpha = 77 # Field of view in degrees

# Initialize FPS calculation
frame_count = 0
start_time = time.time()
fps_current = 0.0

def find_best_detection(detections,
confidence_threshold=0.7):
    best_confidence = 0
    best_box = None
    for detection in detections:
        confidence = detection.Confidence
        if confidence > confidence_threshold and confidence >
best_confidence:
            best_confidence = confidence
            bbox = detection.ROI
            best_box = [bbox.Left, bbox.Top, bbox.Right,
bbox.Bottom]
    return best_box

# Main loop
while True:
    frame_right = cap_right.read()
    frame_left = cap_left.read()

    if frame_right is None or frame_left is None:
        continue

    # Calculate FPS
    frame_count += 1

```

```

if frame_count % 10 == 0:
    elapsed_time = time.time() - start_time
    fps_current = frame_count / elapsed_time
    start_time = time.time()
    frame_count = 0

# Convert frames to CUDA format for Jetson Inference
img_right_cuda = jetson.utils.cudaFromNumpy(frame_right)
img_left_cuda = jetson.utils.cudaFromNumpy(frame_left)

# Perform object detection on the right image
detections_right = net.Detect(img_right_cuda)
detections_left = net.Detect(img_left_cuda)

box_right = find_best_detection(detections_right)
box_left = find_best_detection(detections_left)

if box_right is not None and box_left is not None:
    # Calculate depth of object
    center_right = ((box_right[0] + box_right[2]) // 2,
(box_right[1] + box_right[3]) // 2)
    center_left = ((box_left[0] + box_left[2]) // 2,
(box_left[1] + box_left[3]) // 2)
    depth = tri.find_depth(center_right, center_left,
frame_right, frame_left, B, f, alpha)

    # Draw bounding boxes and information
    cv2.rectangle(frame_right, (box_right[0],
box_right[1]), (box_right[2], box_right[3]), (0, 255, 0), 2)
    cv2.rectangle(frame_left, (box_left[0], box_left[1]),
(box_left[2], box_left[3]), (0, 255, 0), 2)
    cv2.putText(frame_right, f"Distance: {depth:.3f} m",
(200, 50), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (124, 252, 0), 2)
    cv2.putText(frame_left, f"Distance: {depth:.3f} m",
(200, 50), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (124, 252, 0), 2)
    else:
        cv2.putText(frame_right, "TRACKING LOST", (75, 50),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)
        cv2.putText(frame_left, "TRACKING LOST", (75, 50),
cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)

# Display FPS
cv2.putText(frame_right, f"FPS: {fps_current:.1f}", (10,
50), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)

```

```

    cv2.putText(frame_left, f"FPS: {fps_current:.1f}", (10,
50), cv2.FONT_HERSHEY_SIMPLEX, 0.7, (255, 255, 255), 2)

# Show frames
cv2.imshow("frame right", frame_right)
cv2.imshow("frame left", frame_left)

# Break loop on 'q' key press
if cv2.waitKey(1) & 0xFF == ord('q'):
    break

# Release resources
cap_right.release()
cap_left.release()
cv2.destroyAllWindows()

```

How to Execute

1. **Set up your Jetson Nano:** Follow the official [Jetson Nano setup guide](#) to prepare your Jetson Nano with the necessary OS and dependencies.
2. **Install Required Libraries:** Ensure you have the required Python libraries installed. Run the following commands:

```
pip install opencv-python numpy
```

3. **Clone or Copy the Project:** Ensure you have the following files in your project directory:

- `Stereo.py`
- `triangulation.py`

4. **Run the Main Script:** Execute the main script:

```
python Stereo.py
```

5. **Terminate the Program:** To stop the program, press the 'q' key.

Notes

- Ensure both cameras are properly connected and configured.
- Adjust the camera settings (resolution, FPS) as needed based on your specific hardware setup.
- This script is intended for a Jetson Nano environment; ensure all Jetson-specific libraries are correctly installed and configured.

